

INHOUDSOPGAVE

PRIORITEIT	KWALITEITSDOEL	REDEN
HOOG	Onderhoudbaarheid	Het project moet begrijpelijk en aanpasbaar blijven tijdens ontwikkeling en beoordeling.
HOOG	Correctheid	Schaakregels, authenticatiestromen en toestandsovergangen moeten consistent werken.
HOOG	Beveiliging	Wachtwoorden, tokens en beveiligde endpoints moeten veilig worden behandeld.
HOOG	Scheiding van verantwoordelijkheden	Elk project moet een duidelijke verantwoordelijkheid hebben om architectuurvervuiling te voorkomen.
GEMIDDELD	Testbaarheid	Belangrijke logica moet verifieerbaar zijn via geautomatiseerde tests en API-tests.

PRIORITEIT	KWALITEITSDOEL	REDEN
GEMIDDELD	Bruikbaarheid	De interface moet kernacties zoals registratie, inloggen en zetten indienen eenvoudig maken.

1.3 STAKEHOLDERS

STAKEHOLDER	BELANG IN HET SYSTEEM
Eindgebruikers	Willen een duidelijke en betrouwbare manier om online te schaken.
Studentontwikkelaar	Heeft een schone architectuur nodig die realistisch te bouwen, te verklaren en uit te breiden is.
Docenten & beoordelaars	Hebben inzicht nodig in architectuurrenening, scheiding van verantwoordelijkheden en technische onderbouwing.
Toekomstige beheerders	Hebben een systeem nodig dat gemakkelijk te begrijpen, debuggen en door te ontwikkelen is zonder kernregels te breken.

2.1 TECHNISCH

BEPERKING	IMPACT OP DE ARCHITECTUUR
React + TypeScript frontend	De client is geïmplementeerd als een aparte webapplicatie en communiceert uitsluitend via HTTP.
ASP.NET Core Web API backend	Bedrijfslogica en API-endpoints zijn gecentraliseerd in een toegewijd backendproject.
Entity Framework Core	Databasetoegang wordt uitsluitend afgehandeld via EF Core in de data/infrastructuurlaag.
JWT-gebaseerde authenticatie	Beveiligde endpoints zijn afhankelijk van op tokens gebaseerde autorisatie in plaats van serversessies.
Aparte projecten	Frontend, backend en data/infrastructuur blijven gescheiden om architectuurgrenzen te bewaken.

2.2 ORGANISATORISCH

Het project wordt ontwikkeld in een **onderwijscontext** en vereist daarom verklaarbare ontwerpbeslissingen. De beschikbare ontwikkeltijd is beperkt, waardoor de scope realistisch moet blijven. De documentatie is geschreven voor docenten en medestudenten. Het systeem moet demonstraties, testen en incrementele oplevering ondersteunen.

2.3 CONVENTIES

Frontend en backend communiceren uitsluitend via **REST/HTTP met JSON-payloads**. De front-end heeft geen directe toegang tot de database. De backend geeft nooit EF

Core-entiteiten direct terug - antwoorden worden gemapt naar DTO's. C4-diagrammen en een arc42-achtige documentstructuur worden gebruikt om architectuurbeslissingen toe te lichten.

3

Context



Bedrijfscontext: Gebruikers spelen online schaak via de browser. SharpChess gebruikt externe diensten voor e-mailverificatie en aanvullende schaakfuncties. Gebruikers communiceren met de front-end; de back-end valideert de schaakzetten en gebruikt een externe e-mailservice voor accountverificatie.

Implementatiecontext: De applicatie kan lokaal via containers draaien, of afzonderlijk in een cloudomgeving worden uitgerold.

4

Oplossingsstrategie



SharpChess heeft een klassieke web-architectuur met een strikte scheiding van verantwoordelijkheden. Het systeem bestaat uit drie delen.

FRONTEND

React + TypeScript
single-page applicatie.
Toont de interface en
verstuur acties naar de
backend.

BACKEND

ASP.NET Core Web API
met applicatie- en
domeinlogica. Valideert
acties en spelregels.

DATA / INFRASTRUCTUUR

EF Core-configuratie,
repositories,
databasemigraties en
externe service-
implementaties.

De back-end bevat de schaaklogica en is de **enige bron van waarheid**. De front-end en backend communiceren via REST en JSON, met JWT voor authenticatie.

5

Bouwblokweergave



5.7 AUTHENTICATIEMODULE



Registratiestroom

Aanmaken van nieuwe accounts.



E-mailverificatie

Eenmalige tokens met verloopafhandeling.



Inlogstroom

Tokens uitsluitend aan geldige en geverifieerde gebruikers.



Wachtwoordverwerking

Veilige hashing in plaats van opslag als platte tekst.

5.8 SPELMODULE



Bordtoestandsophaling

Huidige spelstatus ophalen voor de frontend.



Zetindiening & -validatie

Controle of gevraagde zet legaal is.



Speltoestandsvoortgang

Voortgang en resultaatafhandeling.



Persistentie

Opslag van partijeninformatie via backend-abstracties.

6

Runtimeweergave



Registratiestroom: Bij registratie stuurt de frontend de ingevulde gegevens naar de backend. De backend valideert het verzoek, hasht het wachtwoord, slaat de gebruiker op en verstuurt een verificatie-e-mail. De frontend toont daarna het resultaat.

7 Implementatieweergave



De codebasis is opgesplitst in afzonderlijke projecten voor API, applicatie, domein, infrastructuur en tests. Deze indeling dwingt architectuurgrenzen af en houdt afhankelijkheden controleerbaar.

8 Concepten



DTO's

Afgeschermd contracten tussen client en server.



JWT-authenticatie

Stateless authenticatie voor beveiligde endpoints.



EF Core

Persistentie en mapping naar de relationele database.



Gelaagde architectuur

Scheiding tussen presentatie, applicatie, domein en infrastructuur.

9 Ontwerpbeslissingen



Belangrijk: De backend bewaakt spelregels en beveiliging centraal. Daardoor blijft de frontend dun en is kritieke logica niet afhankelijk van browsergedrag.

10

Kwaliteitsvereisten



10.1 UTILITY TREE

CATEGORIE	SUB-KWALITEIT	BELANG	TOELICHTING
Onderhoudbaarheid	Duidelijke gelaagdheid	HOOG	Verantwoordelijkheden moeten gemakkelijk te lokaliseren en te wijzigen zijn.
Correctheid	Regelhandhaving	HOOG	Illegale zetten en ongeldige stromen moeten betrouwbaar worden geblokkeerd.
Beveiliging	Authenticatieveiligheid	HOOG	Inloggegevens en beveiligde bronnen mogen niet onzorgvuldig worden blootgesteld.

CATEGORIE	SUB-KWALITEIT	BELANG	TOELICHTING
Testbaarheid	Geïsoleerde logica	GEMIDDELD	Kernlogica moet testbaar zijn zonder de UI.
Bruikbaarheid	Duidelijke feedback	GEMIDDELD	Gebruikers moeten begrijpen wat het systeem doet en waarom acties slagen of mislukken.

10.2 KWALITEITSSCENARIO'S

- 1 Wanneer een niet-geverifieerde gebruiker probeert in te loggen, wijst de backend het verzoek af met een duidelijk antwoord dat de status uitlegt.
- 2 Wanneer de frontend een illegale zet stuurt, wijst de backend deze af zonder de opgeslagen spelstatus te beschadigen.
- 3 Wanneer een toekomstige ontwikkelaar de registratielogica moet wijzigen, is de wijziging mogelijk zonder tegelijkertijd frontend-rendercode of persistentiedetails aan te passen.
- 4 Wanneer een onverwachte uitzondering optreedt, logt het systeem voldoende context voor diagnose en stuurt een veilig antwoord terug naar de client.

11.1 FRONTEND-VERBINDING

SharpChess gebruikt expliciete DTO's en API-tests om de contracten tussen frontend en backend te borgen.

11.2 INSPANNING & SCOPE

Een schaakplatform kan gemakkelijk in scope groeien bij functies zoals matchmaking, persistentie, timers en geavanceerde gameplay. Het project vereist bewuste scopebeheersing zodat architectuurkwaliteit niet wordt opgeofferd voor het aantal functies.

11.3 REGELCORRECTHEID & BEVEILIGING

De backend beheert de schaaklogica en authenticatie centraal. Strikte validatie en duidelijke autorisatie borgen de correctheid en veiligheid van het systeem.

12

Woordenlijst



TERM	BETEKENIS IN SHARPCHESS
DTO	Een data transfer object dat wordt gebruikt voor communicatie tussen frontend en backend.
EF Core	Entity Framework Core, gebruikt voor persistentie en databasetoegang in de infrastructuurlaag.
JWT	JSON Web Token gebruikt voor stateless authenticatie en autorisatie.

TERM	BETEKENIS IN SHARPCHESS
Zetvalidatie	De backend-controle die bepaalt of een gevraagde schaakzet legaal is.
Spelstatus	De huidige toestand van een schaakpartij, inclusief bordgegevens en beurtvolgorde.
Verificatietoken	Een eenmalig token dat wordt gebruikt om een nieuw aangemaakt gebruikersaccount te bevestigen.

Fullstack Semester 3 Sprint 2

SharpChess

≡ ↓↑ 🔍

Backlog 5

Back-end: Unit Tests

High

Duurt tussen de 4 en 8 uur

End-to-End Tests

High

Duurt langer dan 24 uur

Restart Policy Toevoegen

Medium

Duurt tussen de 1 en 4 uur

Readme voor SharpChess

Low

Duurt tussen de 8 en 16 uur

Ethics

Low

Duurt tussen de 8 en 16 uur

To-Do: (MAX 4) 4

Raspberry PI hosting van back-end

High

Duurt langer dan 24 uur

Security Toevoegen volgens de ISO standaarden

High

Duurt langer dan 24 uur

LO6: Kernkwaliteit vergelijken met die van vorige semester

High

Duurt tussen de 4 en 8 uur

Database generatie op website

Medium

Duurt tussen de 8 en 16 uur

D

M

A

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

I

Kubernetes Pod Health

This page shows the last committed snapshot of the SharpChess Kubernetes workload. It is generated locally with `kubect1` and then published through DocFX, so the page does not expose direct cluster access, kubeconfig content, or Kubernetes credentials.

Generated at 2026-05-12 20:21 UTC.

- Namespace scope: `default`
- App label selector: `app=sharpchess-api`

This page shows a generated snapshot. It is not direct unrestricted cluster access, and it does not expose kubeconfig or cluster credentials.

Pod Snapshot

Namespace	Deployment	Pod	Ready	Status	Restarts	Age	Node
default	sharpchess-api	sharpchess-api-76d5888f8-2cdcn	1/1 (True)	Running	0	25d 11h	k3d-sharpchess-cluster-server-0
default	sharpchess-api	sharpchess-api-76d5888f8-w6965	1/1 (True)	Running	0	25d 11h	k3d-sharpchess-cluster-server-0

Deployment Snapshot

Namespace	Deployment	Ready	Updated	Available	Age
-	-	No matching deployments found	-	-	-

Recent Warning Events

No warning events were returned for the selected scope.

What This Means

What is a Kubernetes Pod?

A Pod is the smallest deployable unit in Kubernetes. It contains one or more containers that run together on the same node and share networking and storage context.

Why readiness matters

Readiness tells Kubernetes whether a Pod is ready to receive traffic. A Pod can be running but still not be ready. For a deployment, that usually means the application started but is not healthy enough yet to serve requests.

Basic state meanings

- **Running** means the Pod has been scheduled and the containers are running.
- **Pending** means the Pod has not started properly yet, often because of scheduling, image pull, or storage issues.
- **CrashLoopBackOff** means a container keeps starting and crashing.
- **Ready** means the Pod passed its readiness checks and should be able to handle traffic.

Regenerate Locally

Run the snapshot script from the repository root:

```
K8S_NAMESPACE=default APP_LABEL=sharpchess-api bash scripts/generate-kubernetes-pod-health.sh
```

The script uses `KUBECONFIG` if it is set, or falls back to `~/.kube/config` on your machine.

Why This Page Exists

This page gives deployment monitoring evidence without giving the static DocFX site direct cluster access. The snapshot is generated on a machine that can reach the cluster, committed to the repository, and then published with the rest of the documentation.

Troubleshooting

- **Pending**: check node capacity, image pull access, PVC binding, and whether the namespace exists.
- **ImagePullBackOff**: check the image name, tag, registry access, and any pull secrets.
- **CrashLoopBackOff**: check container logs, startup configuration, environment variables, and database connectivity.
- **NotReady**: check readiness probes, dependency health, migrations, and whether the service is listening on the expected port.